Day 6 — Elastic Stack (ELK) Fundamentals + Filebeat

Name:	Mohammed Farhan
Role:	Information Security (Intern)
Organization:	CyberShelter UAE
Date:	4 July 2026

Objective

To deploy a functional Elastic Stack (Elasticsearch, Logstash, Kibana) alongside an existing Wazuh deployment, configure Filebeat to ship Linux auditd and nginx access logs into Elasticsearch, build a Kibana dashboard visualizing key security events, and translate five working Splunk SPL detections into equivalent Kibana Query Language (KQL) searches to demonstrate cross-platform SIEM fluency.

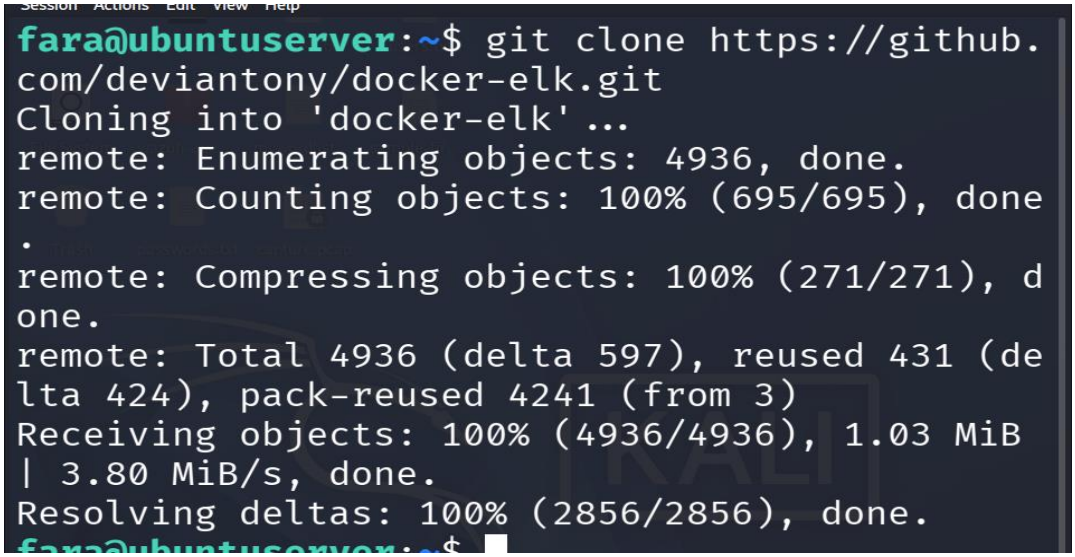
System Information

Detail	**Value**
Host	Ubuntu Server 22.04 LTS — 192.168.56.104
Deployment Method	Docker Compose (deviantony/docker-elk)
Elastic Stack Version	9.4.2
Log Sources Shipped	auditd (`/var/log/audit/audit.log`), nginx access logs, Linux auth.log

Part 1 — ELK Stack Deployment (Full Steps)

Step 1 — Clone the Repository

- `git clone https://github.com/deviantony/docker-elk.git`



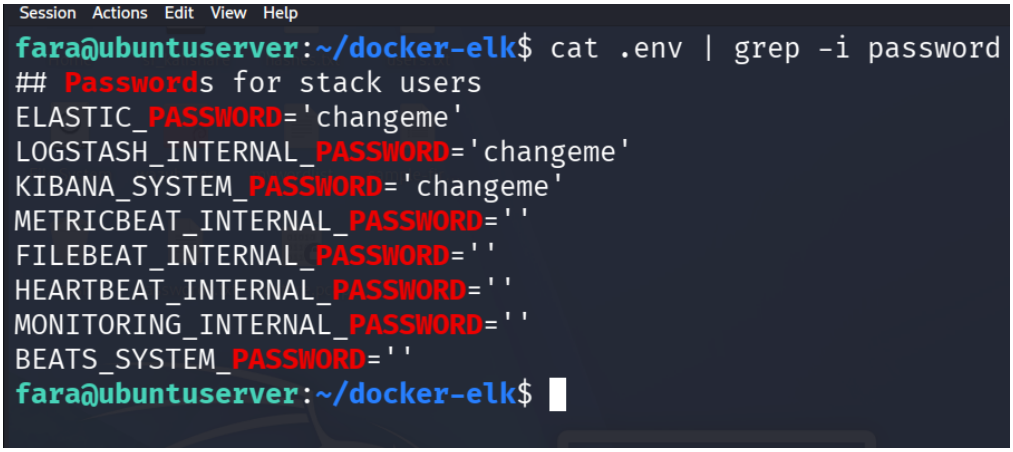
- `cd docker-elk`

What this does:

Downloads deviantony's official pre-configured Docker Compose project for deploying Elasticsearch, Logstash, and Kibana together, and moves into the project directory.

Step 2 — Check Default Credentials

- `cat .env | grep -i password`



Confirmed default credential set as `ELASTIC_PASSWORD='changeme'` in the `.env` file, used for the `elastic` superuser account.

Step 3 — Resolve Port Conflict

Since Wazuh's Indexer already occupied port 9200 on the same host (both being Elasticsearch-based technologies), the compose file was edited to avoid conflict:

- `nano docker-compose.yml`

Changed:

```
- 9200:9200
- 9300:9300
```

To:

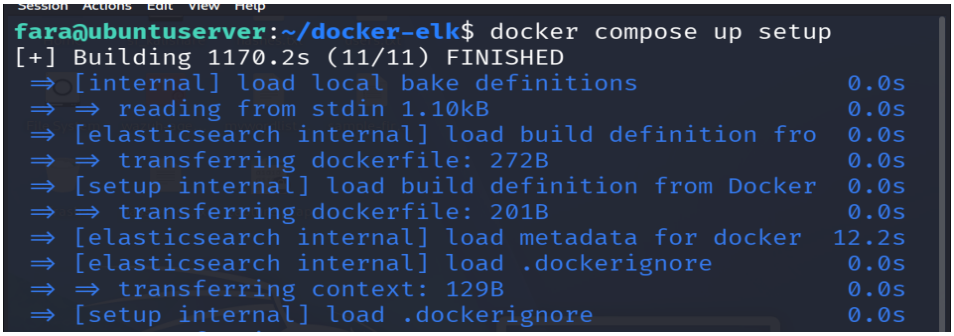
```
- 9201:9200
- 9301:9300
```

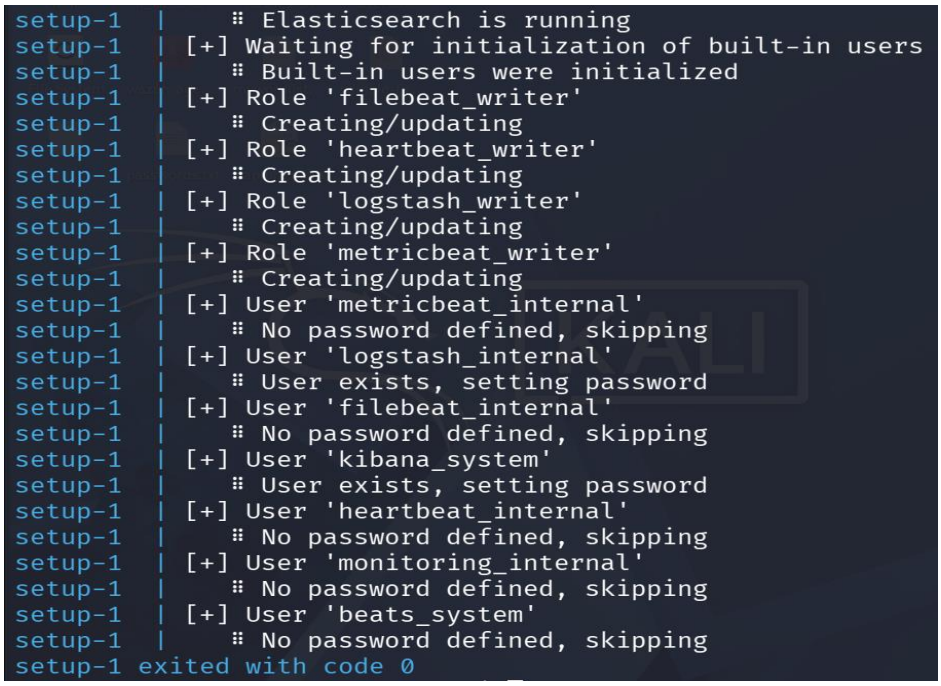
Kibana's default port mapping (5601:5601) was left unchanged, as it did not conflict with any existing service.

Step 4 — Run Initial Setup Service

- `docker compose up setup`

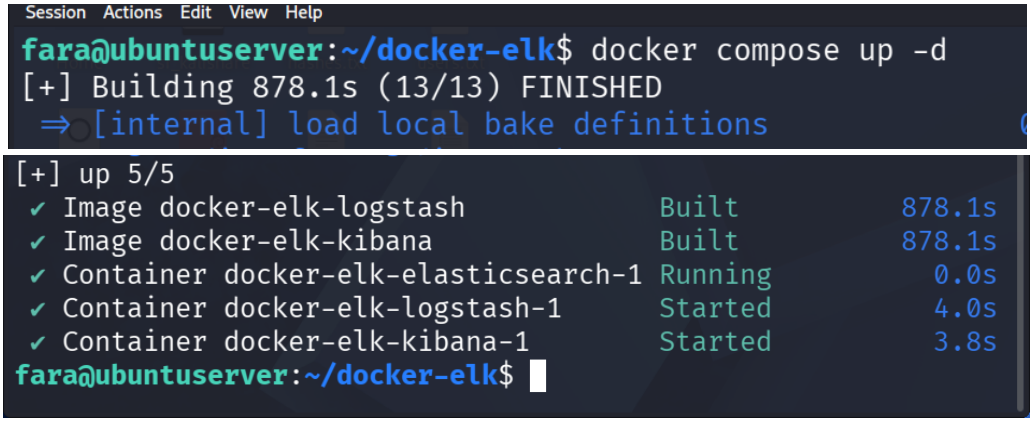
What this does: Runs a one-off container that initializes Elasticsearch's built-in users (logstash_internal, kibana_system) and required roles using the credentials defined in `.env`. This step must run once before starting the main stack.





Step 5 — Start the Full Stack

- `docker compose up -d`



What this does: Builds and starts all three containers (Elasticsearch, Logstash, Kibana) in detached mode.

Step 6 — Verify Containers Running

- `docker ps`

Confirmed all three ELK containers running alongside the three existing Wazuh containers, with no port conflicts:

- `docker-elk-elasticsearch-1` — ports 9201, 9301
- `docker-elk-logstash-1` — ports 5044, 9600, 50000
- `docker-elk-kibana-1` — port 5601

Step 7 — Open Firewall Port

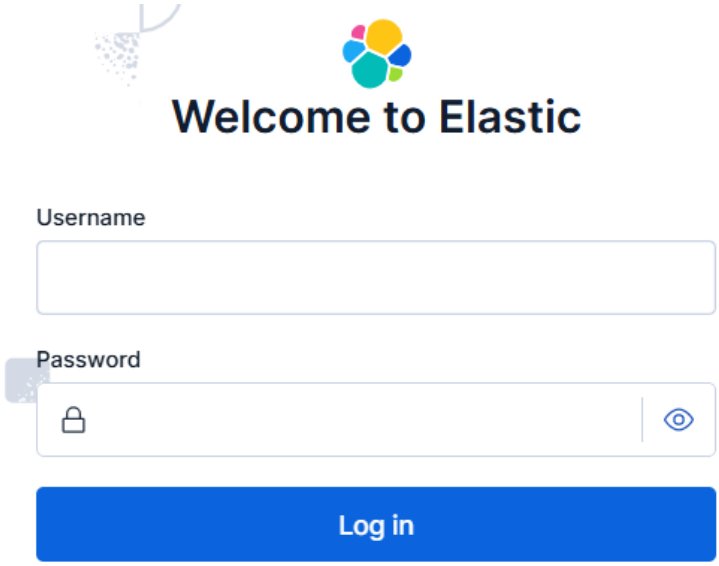
- `sudo ufw allow 5601/tcp`

What this does: Allows external access to Kibana's web interface through UFW, which blocks all traffic by default per Day 1 hardening configuration.

Step 9 — Access Kibana

Kibana was accessed at:

- http://192.168.56.104:5601



Login confirmed successful using:

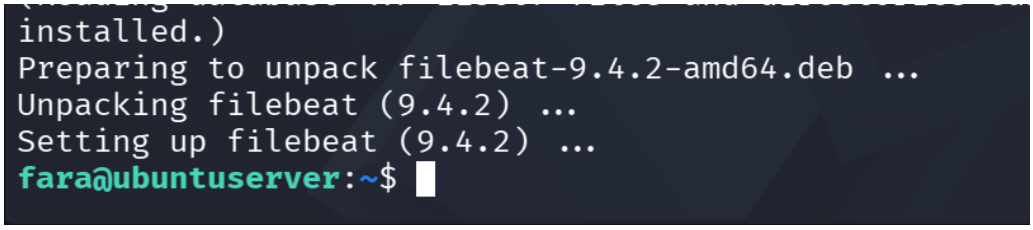
- `Username: elastic`
  `Password: changeme`

Part 2 — Filebeat Configuration

Filebeat 9.4.2 was downloaded and installed to match the Elastic Stack version:

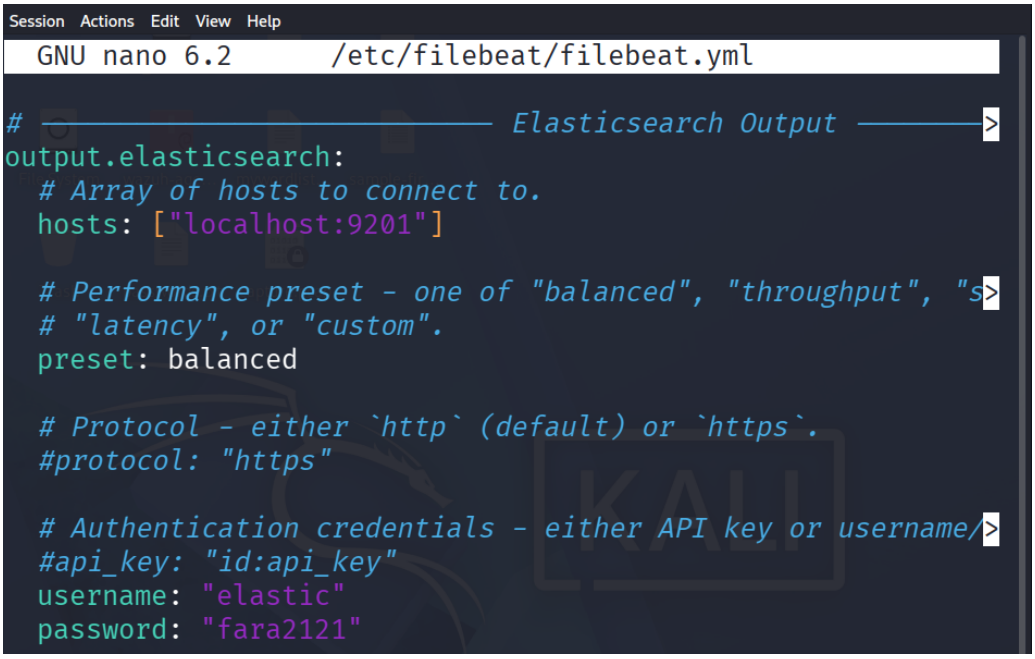
- `curl -L -O` https://artifacts.elastic.co/downloads/beats/filebeat/filebeat-9.4.2-amd64.deb

- `sudo dpkg -i filebeat-9.4.2-amd64.deb`



Output Configuration

- `sudo nano /etc/filebeat/filebeat.yml`

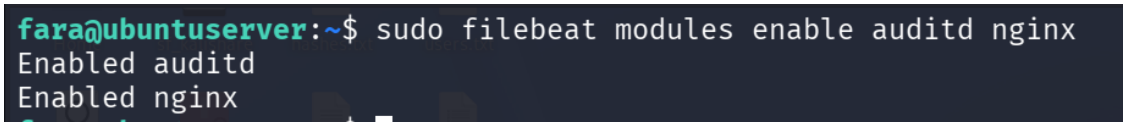


Configured to point at the remapped Elasticsearch port with authentication:

```
output.elasticsearch:
hosts: ["localhost:9201"]
preset: balanced
username: "elastic”
password: "changeme"
```

Modules Enabled

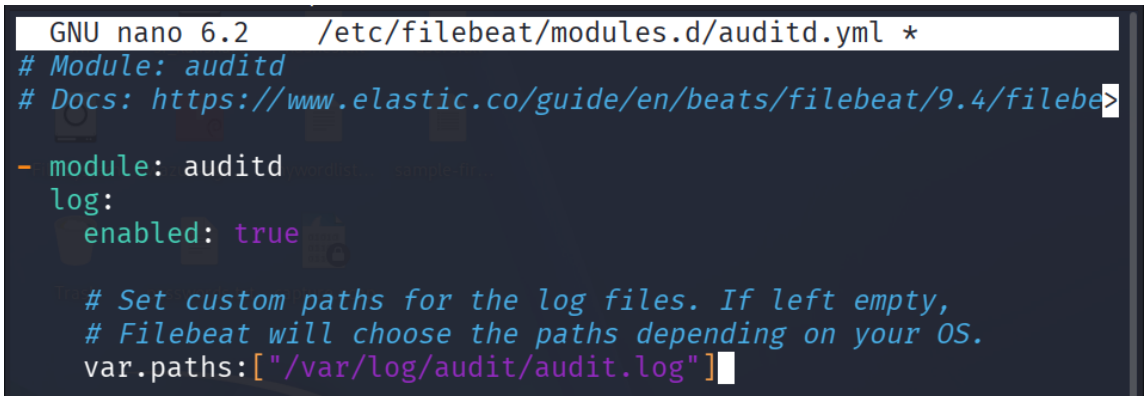
- `sudo filebeat modules enable auditd nginx`



1. Auditd module — enabled and pointed at `/var/log/audit/audit.log`:

yaml

```
- module: auditd
  log:
  enabled: true
  var.paths: ["/var/log/audit/audit.log"]
```



2. Nginx module — nginx was installed to generate real access log data

- (`sudo apt install nginx -y`),

```
fara@ubuntuserver:~$ sudo apt install nginx -y
Reading package lists ... Done
Building dependency tree ... Done
Reading state information ... Done
The following additional packages will be installed:
  fontconfig-config fonts-dejavu-core libdeflate0
  libfontconfig1 libgd3 libjbig0 libjpeg-turbo8
  libjpeg8 libnginx-mod-http-geoip2
  libnginx-mod-http-image-filter
  libnginx-mod-http-xslt-filter libnginx-mod-mail
  libnginx-mod-stream libnginx-mod-stream-geoip2
  libtiff5 libwebp7 libxpm4 nginx-common nginx-core
Suggested packages:
  libgd-tools fcgiwrap nginx-doc ssl-cert
The following NEW packages will be installed:
  fontconfig-config fonts-dejavu-core libdeflate0
  libfontconfig1 libgd3 libjbig0 libjpeg-turbo8
  libjpeg8 libnginx-mod-http-geoip2
  libnginx-mod-http-image-filter
  libnginx-mod-http-xslt-filter libnginx-mod-mail
  libnginx-mod-stream libnginx-mod-stream-geoip2
  libtiff5 libwebp7 libxpm4 nginx nginx-common
  nginx-core
0 upgraded, 20 newly installed, 0 to remove and 2 not
```

Verified nginx was running

- sudo systemctl status nginx

```
Session  Actions  Edit  View  Help
fara@ubuntuserver:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a>
     Loaded: loaded (/lib/systemd/system/nginx.servic>
     Active: active (running) since Sun 2026-07-05 12>
       Docs: man:nginx(8)
    Process: 10274 ExecStartPre=/usr/sbin/nginx -t -q>
    Process: 10275 ExecStart=/usr/sbin/nginx -g daemo>
   Main PID: 10371 (nginx)
      Tasks: 3 (limit: 6974)
     Memory: 7.1M
        CPU: 43ms
     CGroup: /system.slice/nginx.service
             ├─10371 "nginx: master process /usr/sbin>
             ├─10373 "nginx: worker process" "" "" "">
             └─10374 "nginx: worker process" "" "" "">
```

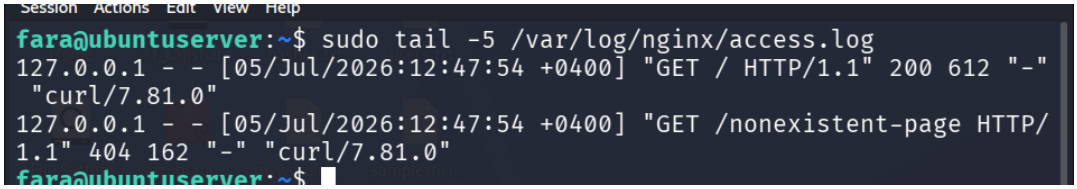
Generated real test traffic

- `curl http://localhost`
  `curl http://localhost/nonexistent-page`

This produced one **200 OK** (default nginx welcome page) and one **404 Not Found** — giving us genuine varied log data instead of an empty log file.

Verified nginx actually wrote to its access log

- `sudo tail -5 /var/log/nginx/access.log`

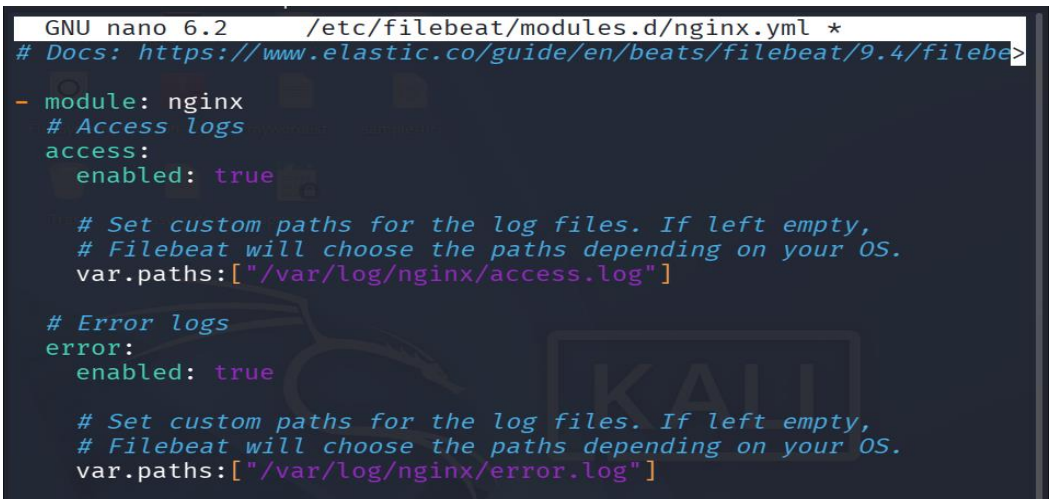


Enabled nginx module in Filebeat

- `sudo filebeat modules enable auditd nginx`

Configured the nginx module

```
module: nginx
 access:
 enabled: true
 var.paths: ["/var/log/nginx/access.log"]
 error:
enabled: true
var.paths: ["/var/log/nginx/error.log"]
```



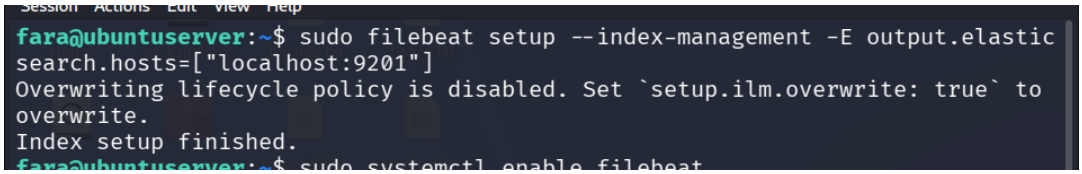
3. Custom filestream input — a dedicated input was added for `/var/log/auth.log`, since SSH authentication events are logged via PAM to this file rather than through auditd:

yaml

```
- type: filestream
  id: auth-log-input
  enabled: true
  paths:
- /var/log/auth.log
```

Index Setup

- `sudo filebeat setup --index-management -E output.elasticsearch.hosts=["localhost:9201"]`

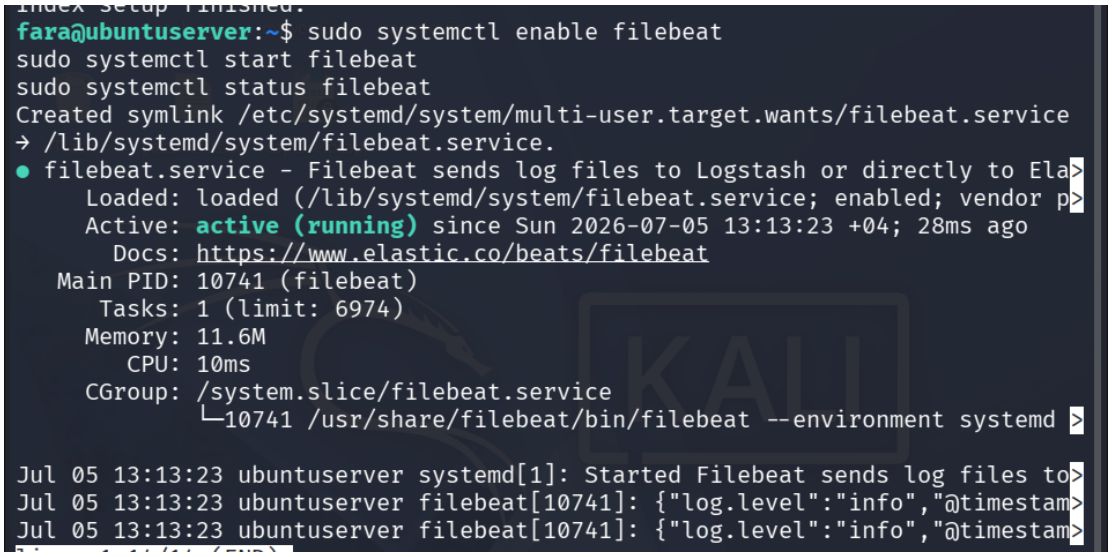


Output confirmed: Index setup finished.

Service Enabled and Started

- `sudo systemctl enable filebeat`
- `sudo systemctl start filebeat`
- `sudo systemctl status filebeat`

Confirmed `Active: active (running)`.



Data Verification

- `curl -u elastic:changeme "http://localhost:9201/_cat/indices?v" | grep filebeat`

Confirmed Filebeat index created with over 40,000 documents indexed and growing, verifying the full pipeline from log source through Filebeat to Elasticsearch.

Part 3 — Kibana Dashboard

Data View Creation

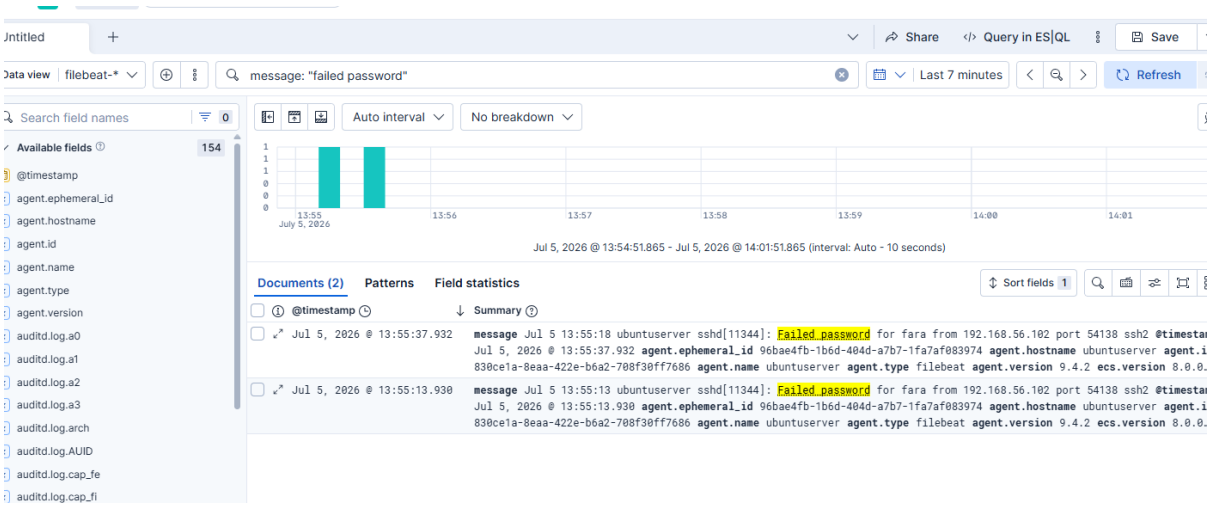
Navigated to **Stack Management → Data Views → Create data view**, configured:

- Name: `filebeat-*`
- Index pattern: `filebeat-*`
- Timestamp field: `@timestamp`

Visualizations Built

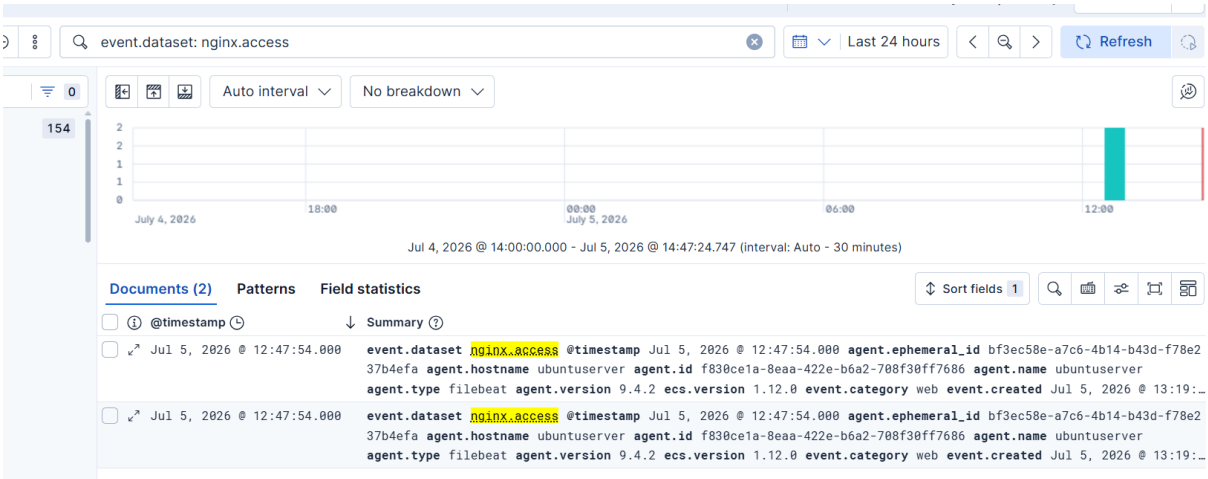
Three visualizations were built in Kibana Lens and combined into a single dashboard:

1. Failed SSH Attempts Over Time —

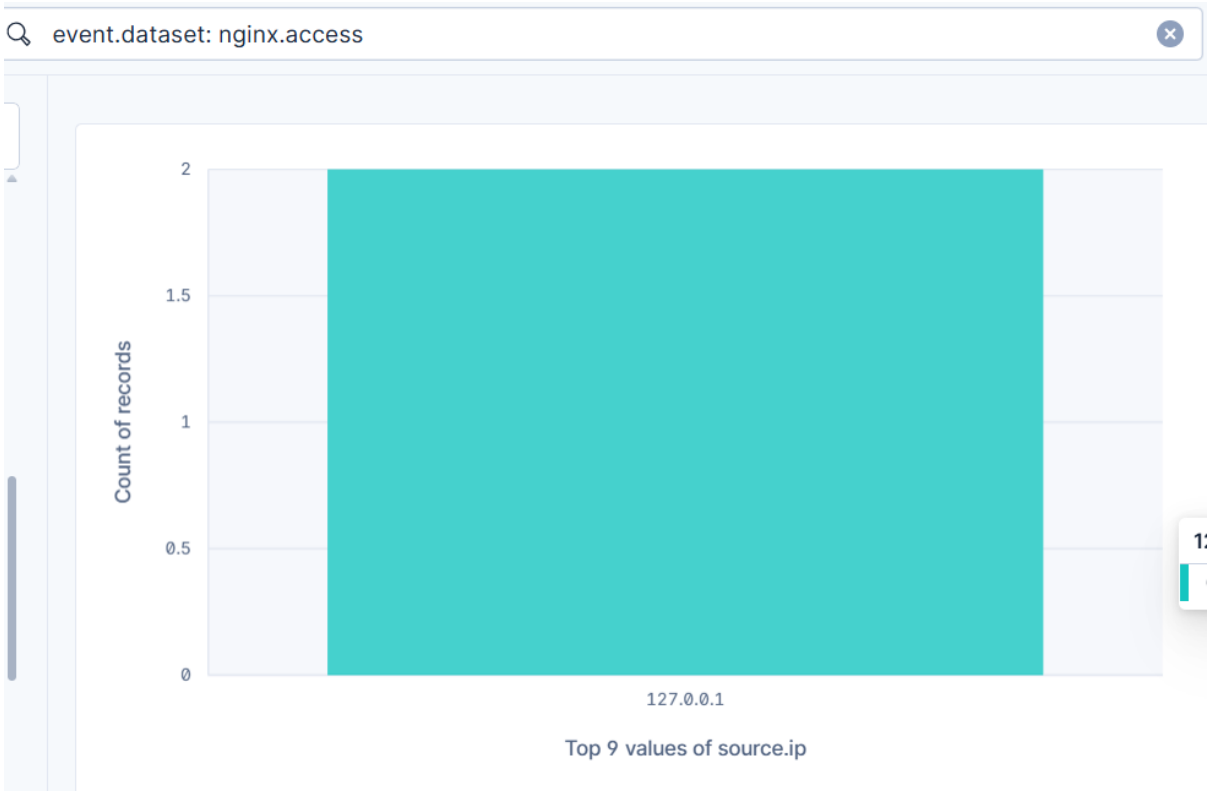


Bar chart filtering `message: "Failed password"`, showing real brute-force test attempts correctly indexed with timestamps.

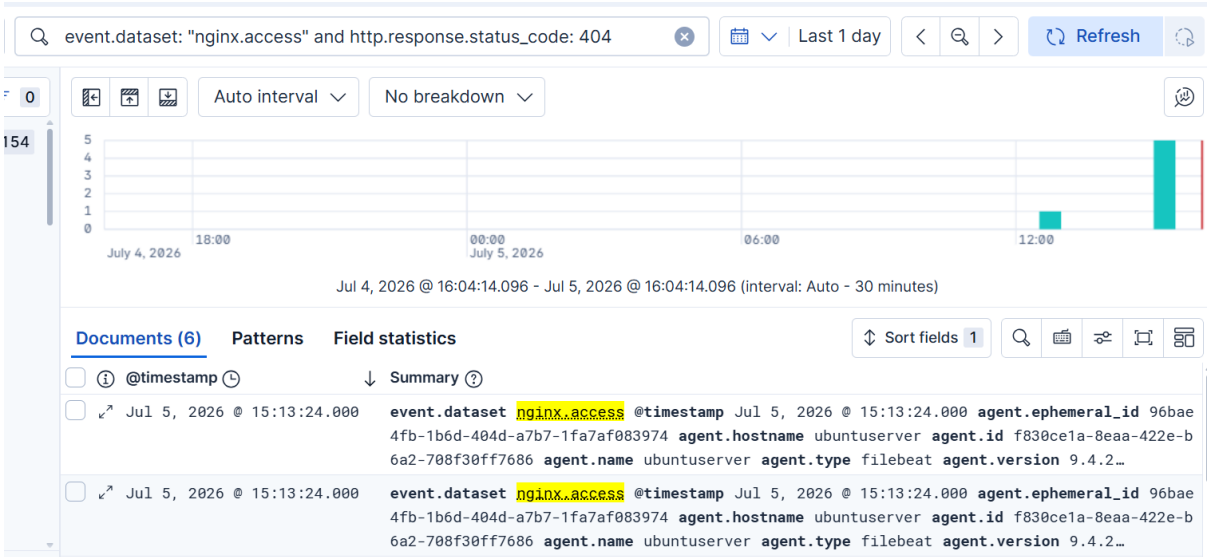
2. Top Source IPs (nginx access) —



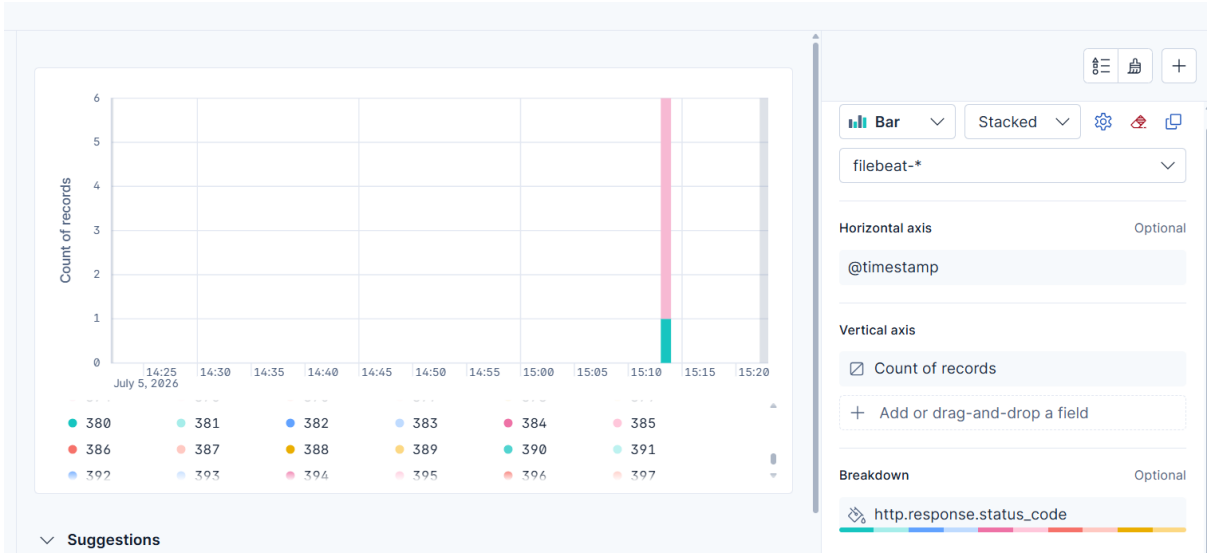
Bar chart using the `source.ip` field from nginx access logs, filtered by `event.dataset: nginx.access`.

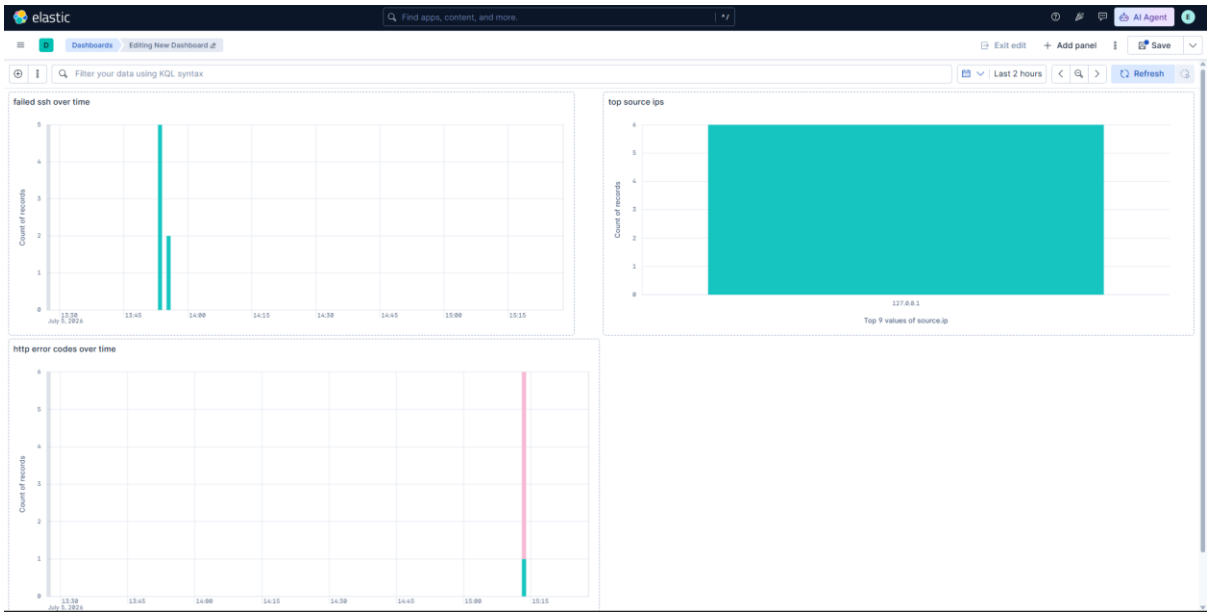


3.HTTP Error Codes Over Time —



Bar chart broken down by `http.response.status_code`, distinguishing successful (200) and not-found (404) requests generated via test traffic including probes of common attack-surface paths (`/admin`, `/wp-admin`, `/login`).





Known Limitation — GeoIP Enrichment

The "Top countries by source IP" requirement could not be fully demonstrated as originally scoped: nginx traffic originated exclusively from `127.0.0.1` (localhost test requests), a private IP address with no real-world geographic mapping.

Part 4 — SPL to KQL Comparison Table

Detection

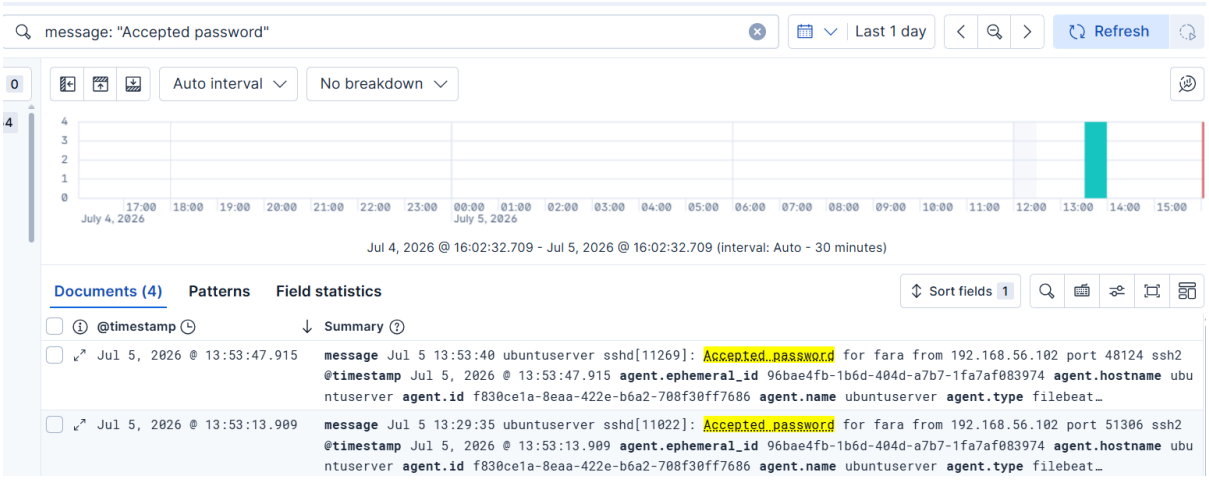
1.Failed SSH Login
2.Successful SSH Logins
3.HTTP 404 Not Found Errors
4.HTTP 200 Successful Requests
5.Auditd Process Activity

Splunk SPL	Kibana KQL	Plain English
index=main "Failed password"	message: "Failed password"	Finds all failed SSH login attempts, indicating possible brute force activity
index=main "accepted password"	message: "Accepted password"	Finds successful SSH logins to establish a baseline of legitimate access
index=main sourcetype=ufw_firewall "404"	event.dataset: "nginx.access" and http.response.status_code: 404	Finds web requests to non-existent pages
index=main sourcetype=ufw_firewall "200"	event.dataset: "nginx.access" and http.response.status_code: 200	Finds successful web requests, representing normal baseline traffic
index=main "docker"	auditd.log.key: docker*ARCH*	Finds low-level system process activity captured by the Linux audit subsystem

1.Failed SSH Login

2.Successful SSH Logins

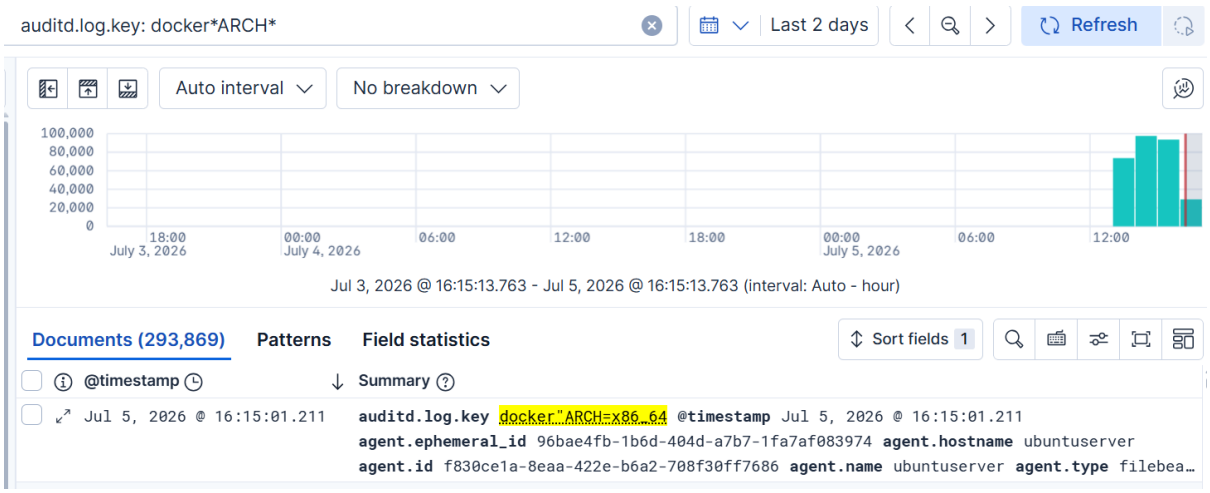
.



3.HTTP 404 Not Found Errors

4.HTTP 200 Successful Requests

5.Auditd Process Activity



Architectural Comparison — Splunk vs Elastic

A key professional distinction was reinforced through this exercise: Splunk is deployed as a single unified binary installer with its own proprietary search language (SPL) built around a piped, command-chaining syntax. Elastic Stack is composed of multiple independent, containerized services (Elasticsearch, Logstash, Kibana) that communicate over the network, using KQL — a filter-first query syntax closer to structured field matching than sequential data transformation. Both approaches achieve equivalent detection outcomes, but require different query construction habits. Since CyberShelter's clients may run either platform, fluency in both is directly applicable to client engagements.

Conclusion

Day 6 successfully established a working Elastic Stack deployment coexisting with existing Wazuh and Splunk installations on the same host. Filebeat was configured to ship both structured module-based data (auditd, nginx) and a custom filestream input (auth.log), with a real YAML configuration error diagnosed and corrected via service logs. Three Kibana visualizations were built and combined into a functional SOC dashboard, and five SPL detections were successfully translated into working KQL equivalents, each verified against live data. This work directly demonstrates the dual-platform SIEM fluency required to support CyberShelter's diverse client environments.